
Input and output devices and methods

Inputs & Outputs



- Computer communicate with outside world using Basic I/O hardware
 - ports, buses, devices and controllers
- ☐ There are I/O Software to facilitate I/O operations
 - Interrupt Handlers, Device Driver, Device- Independent Software, User-Space I/O Software
- ☐ Important concepts
 - Three ways to perform I/O operations:
 - Polling
 - Interrupt
 - DMAs

Peripheral Devices

- □ I/O Subsystem:
 - Provides an efficient mode of communication between the central system and the outside environment
- □ Peripheral (or I/O Device): □ Input or Output devices attached to the computer
 - □ Monitor (Visual Output Device) : CRT, LCD
 - □ KBD (Input Device) : light pen, mouse, touch screen, joystick, digitizer
 - □ Printer (Hard Copy Device) : Dot matrix (impact), thermal, ink jet, laser (non-impact)
 - □ Storage Device : Magnetic tape, magnetic disk, optical disk
- □ ASCII (American Standard Code for Information Interchange) Alphanumeric Characters
 - I/O communications are usually involved in the transfer of ASCII information
 - □ ASCII Code :
 - 7 bit : 00 - 7F (0 - 127)
 - 80 - FF (128 - 255) : Greek, Italic, Graphics,

.

I/O Organisation

- Input-Output Interface

- Interface
 - A conversion of signal values may be required
- A synchronization mechanism may be needed
 - The data transfer rate of peripherals is usually slower than the transfer rate of the CPU
- Data codes and formats in peripherals differ from the word format in the CPU and Memory
- The operating modes of peripherals are different from each other
 - Each peripherals must be controlled so as not to disturb the operation of other peripherals connected to the CPU

Interface:

- Special hardware components between the CPU and peripherals
- Supervise and Synchronize all input and output transfers

I/O Organisation

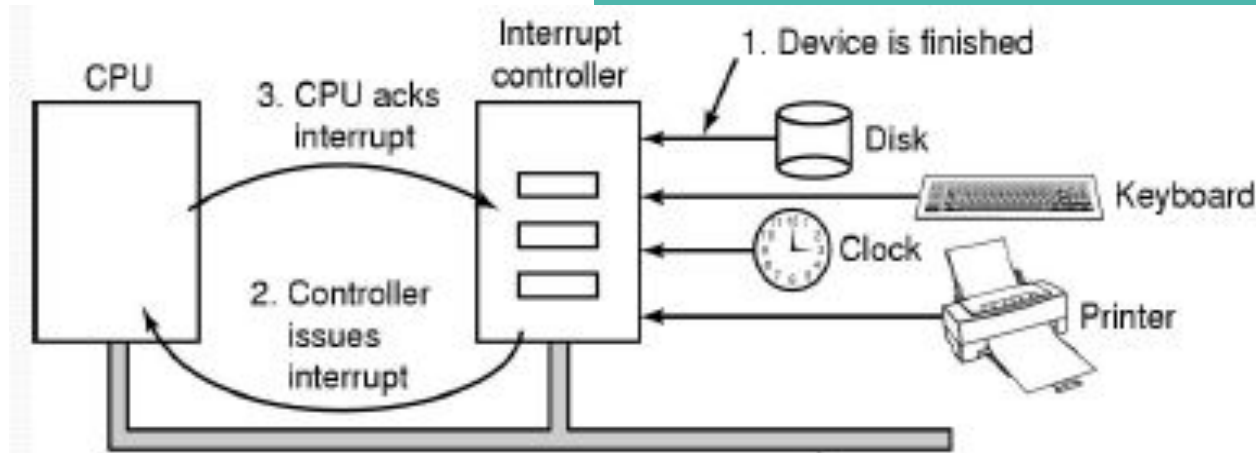
- I/O Bus
 - □ Data lines
 - □ Address lines
 - □ Control lines
- □ Interface Modules :
 - □ SCSI (Small Computer System Interface)
 - □ IDE (Integrated Device Electronics)
 - □ RS-232
- □ I/O command :
 - □ Control Command
 - □ Status Command
 - □ Input Command
 - □ Output Command

Polling

- Polling - use CPU to
 - Busy wait and watch status bits
 - Feed data into a controller register 1 byte at a time
 - EXPENSIVE for large transfers
 - Not acceptable except small dedicated systems not running multiple processes
-

Interrupts

Connections between devices and interrupt controller actually use interrupt lines on the bus rather than dedicated wires



Host-controller interface: Interrupts

- CPU hardware has the interrupt report line that the CPU senses after executing every instruction
 - ☐ device raises an interrupt
 - ☐ CPU catches the interrupt and saves the state (e.g: Instruction pointer)
 - ☐ CPU dispatches the interrupt handler
 - ☐ interrupt handler determines cause, services the device and clears the interrupt
- ☐ Why interrupts? IO devices need attention
- ☐ Real life analogy for interrupt
 - ☐ An alarm sets off when the food is ready
 - ☐ So you can do other things in between

Support for Interrupts

- Need the ability to defer interrupt handling during critical processing
- Need efficient way to dispatch the proper device
 - Interrupt comes with an address (offset in interrupt vector) that selects a specific interrupt handling
- Need multi level interrupts - interrupt priority level

Interrupt Handler

- At boot time, OS probes the hardware buses to
 - determine what devices are present
 - install corresponding interrupt handlers into the interrupt vector
- ☐ During I/O interrupt, controller signals that device is ready

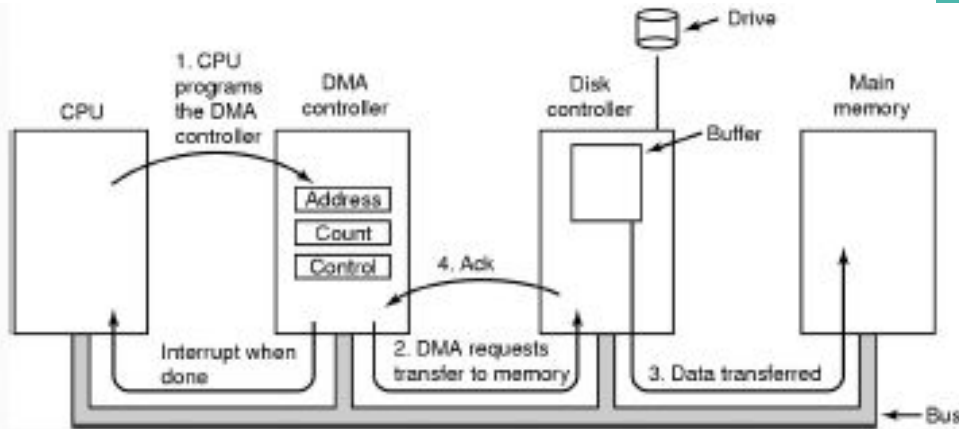
Other Types of Interrupts

Interrupt mechanisms are used to handle wide variety of exceptions:

- ☐ Division by zero, wrong address
- ☐ Virtual memory paging
- ☐ System calls (software interrupts/signals, trap)

Direct Memory Access (DMA)

- Assists in exchange of data between CPU and I/O controller
- CPU can request data from I/O controller byte by byte – but this might be inefficient (e.g. for disk data transfer)
- Uses a special purpose processor, called a DMA controller



Operation of a DMA transfer

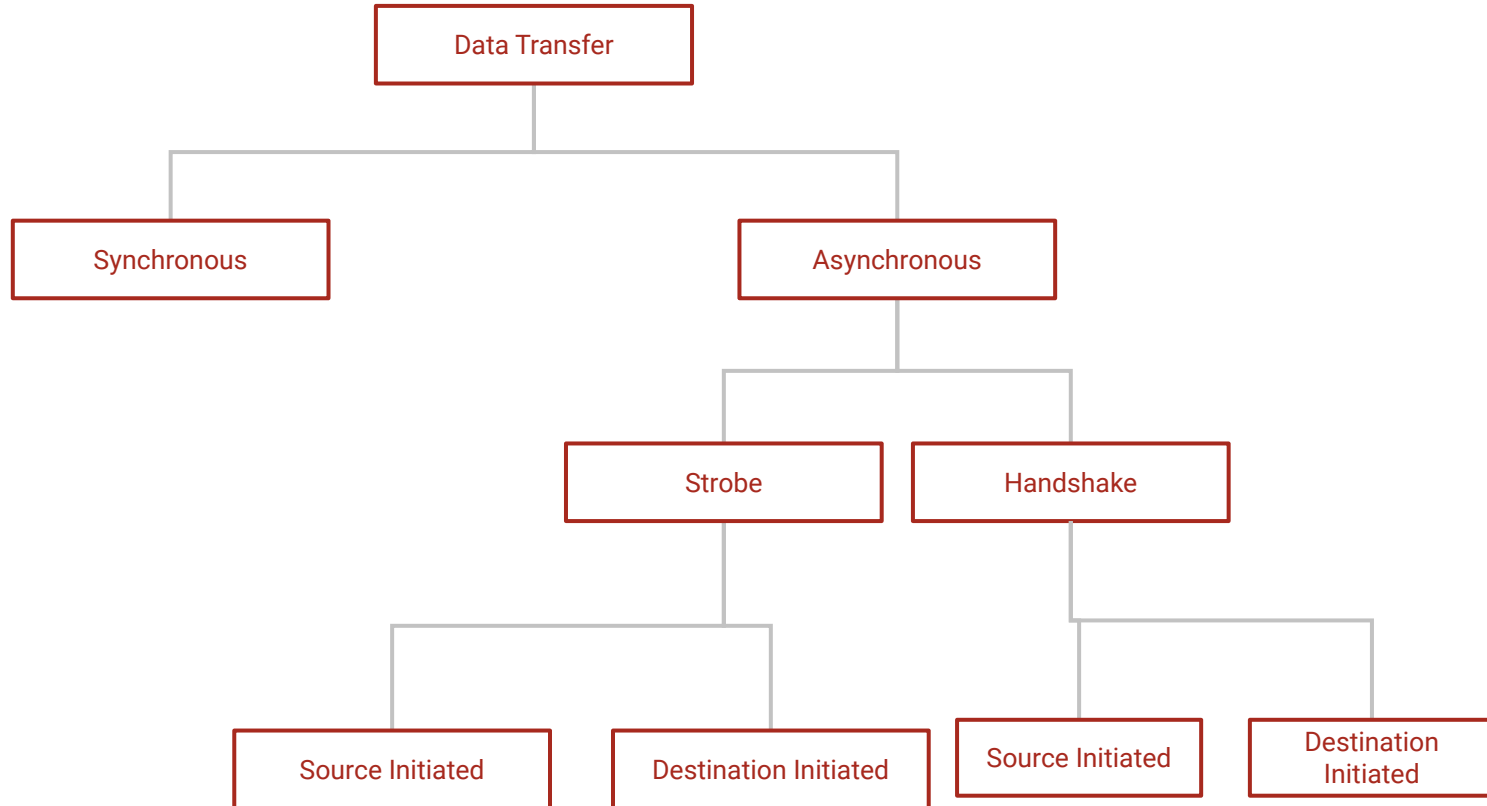
DMA-CPU Protocol

- Use disk DMA as an example
 - CPU programs DMA controller, sets registers to specify source/destination addresses, byte count and control information (e.g., read/write) and goes on with other work
 - DMA controller proceeds to operate the memory bus directly without help of main CPU – request from I/O controller to move data to memory
 - Disk controller transfers data to main memory
 - Disk controller acks transfer to DMA controller
-

DMA Issues

- Handshaking between DMA controller and the device controller
- Cycle stealing
 - DMA controller takes away CPU cycles when it uses CPU memory bus, hence blocks the CPU from accessing the memory
- In general DMA controller improves the total system performance

Data Transfer



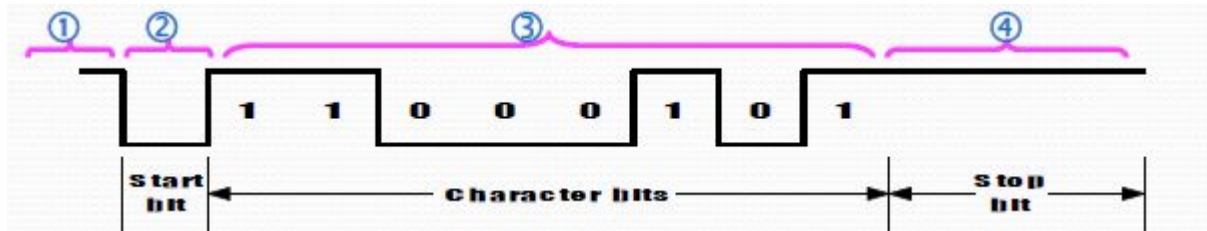
Synchronous and Asynchronous data transfer

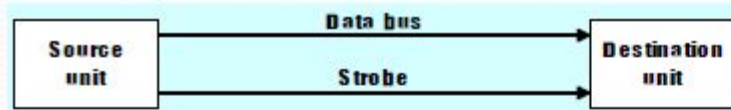
- Synchronous Data Transfer ("occurring at the same time")
 - All data transfers occur simultaneously during the occurrence of a clock pulse. I.e: This is where the sender and receiver are synchronised
 - ☐ Registers in the interface share a common clock with CPU registers, and it is used as a reference point for data transmission, so it occurs at regular intervals.
- ☐ Asynchronous Data Transfer
 - ☐ Internal timing in each unit (CPU and Interface) is independent (does not use a common timing signal)
 - ☐ Each unit uses its own private clock for internal registers (The devices aren't permanently synced, only during transmission)
- Almost all parallel transmission is synchronous
- Asynchronous transmission is used when data is sent sporadically, e.g. mouse/ keyboard

Synchronous	Asynchronous
• Fast transmission • Needs a common clock signal, or some way of sharing it • May have to wait briefly until data can be sent	• Slower transmission, due to the extra bits and the gaps • Cheap and easy to implement = no clock sharing • Can transmit when ready

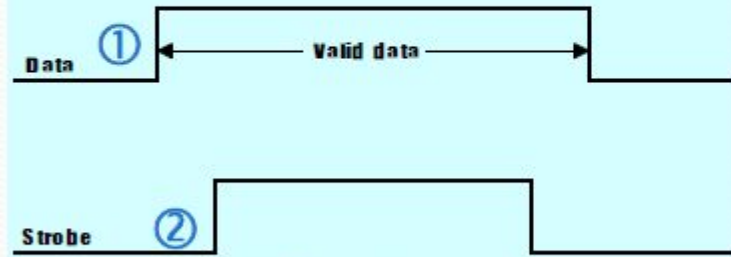
Asynchronous transmission

- Special bits are inserted at both ends of the character code
- Each character consists of three parts :
 - start bit : always "0", indicate the beginning of a character
 - character bits : data
 - stop bit : always "1"
- □ Asynchronous transmission rules :
 - Then a character is not being sent, the line is kept in the 1-state [1]
 - The initiation of a character transmission is detected from the start bit, which is always "0" [2]
 - The character bits always follow the start bit [3]
 - After the last bit of the character is transmitted, a stop bit is detected when the line returns to the 1-state for at least one bit time [4]



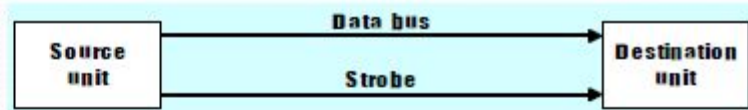


(a) Block diagram

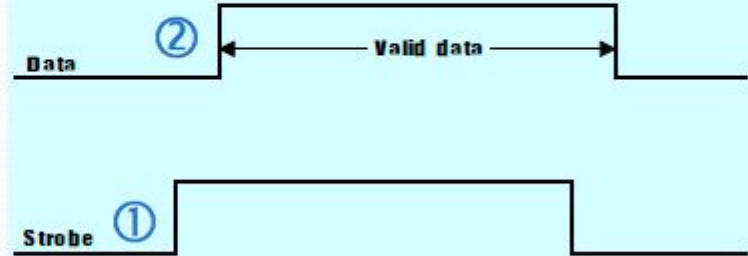


(b) Timing diagram

Source-initiated strobe



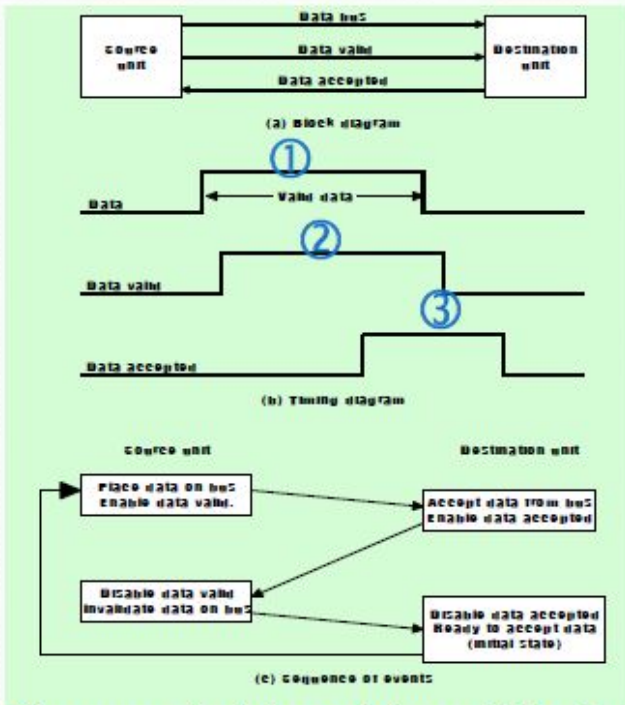
(a) Block diagram



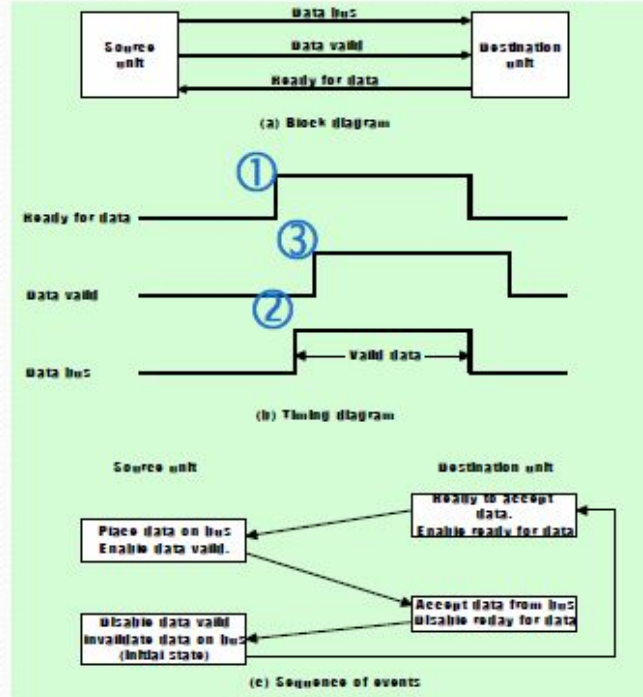
(b) Timing diagram

Destination-initiated strobe

In Strobe data transfer, information is sent in a rapid succession of pulses. It is like sending a signal with a flashlight - the data is transmitted in quick flashes, allowing for quick communication between devices. This type of transfer is often used for quick and simple exchanges of information.



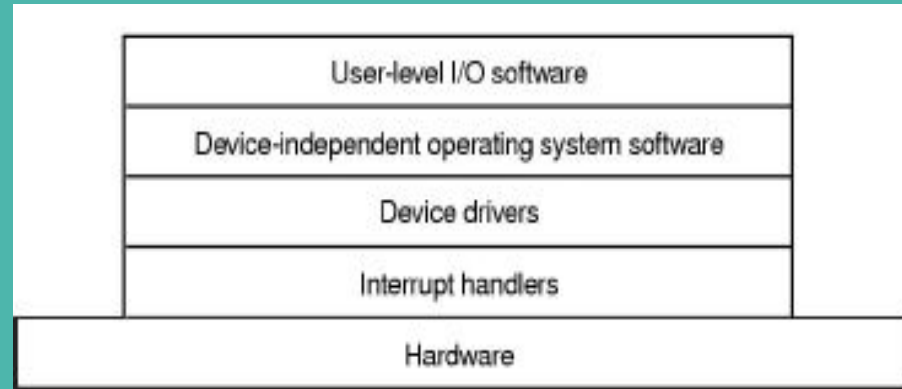
Source-initiated handshake



Destination-initiated handshake

Handshake data transfer involves a more intricate process. Before any data is exchanged, the devices involved go through a series of steps to ensure that they are ready to communicate. It's like a handshake between two people before starting a conversation - there is a protocol to follow before the actual interaction begins. This type of transfer is often used for more complex and secure exchanges of information.

I/O Software Layers



Device Controller

- I/O units typically consist of
 - A mechanical component: the device itself
 - An electronic component: the device controller or adapter.
 - Interface between controller and device is a very low level interface.
 - Example:
 - Disk controller converts the serial bit stream, coming off the drive into a block of bytes, and performs error correction.
-

I/O Controller

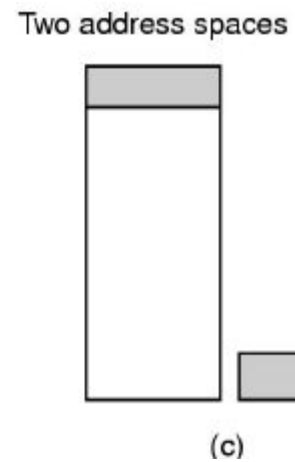
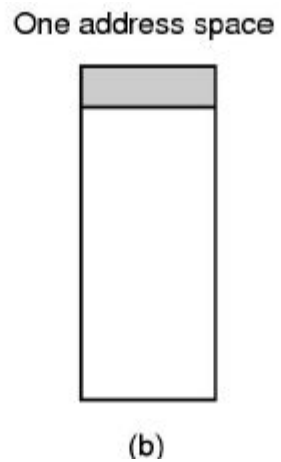
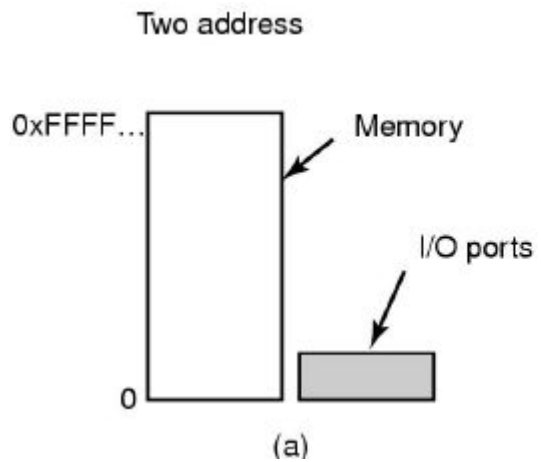
- Disk controller
 - implements the disk side of the protocol that does:
 - bad error mapping, prefetching, buffering, caching
- Controller has registers for data and control
- CPU and controllers communicate via
 - I/O instructions and registers
 - Memory-mapped I/O

I/O Ports

- 4 registers - status, control, data-in, data-out
 - **Status** -
 - This register holds information about the current status of the device connected to the port. It might indicate whether a command has been completed, if there is data available to be read, or if there's an error condition, among other things.
 - states whether the current command is completed, byte is available, device has an error, etc
 - **Control** -
 - host determines to start a command or change the mode of a device
 - The control register allows the host (the main computer system) to initiate commands or change the mode of the connected device. It's used to send instructions to the device.
 - **Data-in** - host reads to get input
 - For example, if the device is a keyboard, when a key is pressed, the corresponding data would be available for the host to read from this register.
 - **Data-out** - host writes to send output
 - For instance, if the device is a printer, the host might write printable data to this register for the printer to receive and print.
- Size of registers - 1 to 4 bytes
 - The size of the registers determines the maximum amount of data that can be transferred or stored at once.

Interfacing Peripherals

- I/O-mapped I/O
 - Use special CPU instructions: Special_instruction Reg, Port
 - In memory-mapped I/O, the addresses used to access memory are the same addresses used to access peripheral devices.
 - Peripheral devices are assigned addresses within the memory address space.
 - Reads and writes to these addresses are interpreted differently by the system. When the CPU accesses an address corresponding to a peripheral device, it triggers a read or write operation on that device rather than accessing actual memory.
 - This approach simplifies programming since the CPU interacts with devices in the same way it interacts with memory.
 - However, it can lead to contention between memory access and I/O access, as both share the same address space.
- Memory-mapped I/O
 - A simpler and more convenient way to interface I/O devices
 - Each device registers is assigned to a memory address in the address space of the microprocessor
 - Use native CPU load/store instructions: LOR/STR Reg, [Reg, #imm]
 - In I/O-mapped I/O, separate instructions are used for accessing memory and I/O devices.
 - I/O devices have their own address space distinct from the memory address space.
 - Special instructions are used to access I/O devices, typically different from those used for accessing memory.
 - This approach allows for better separation between memory and I/O operations, reducing contention between the two.
 - However, it may require more complex programming, as developers need to use different instructions for accessing I/O devices compared to accessing memory.

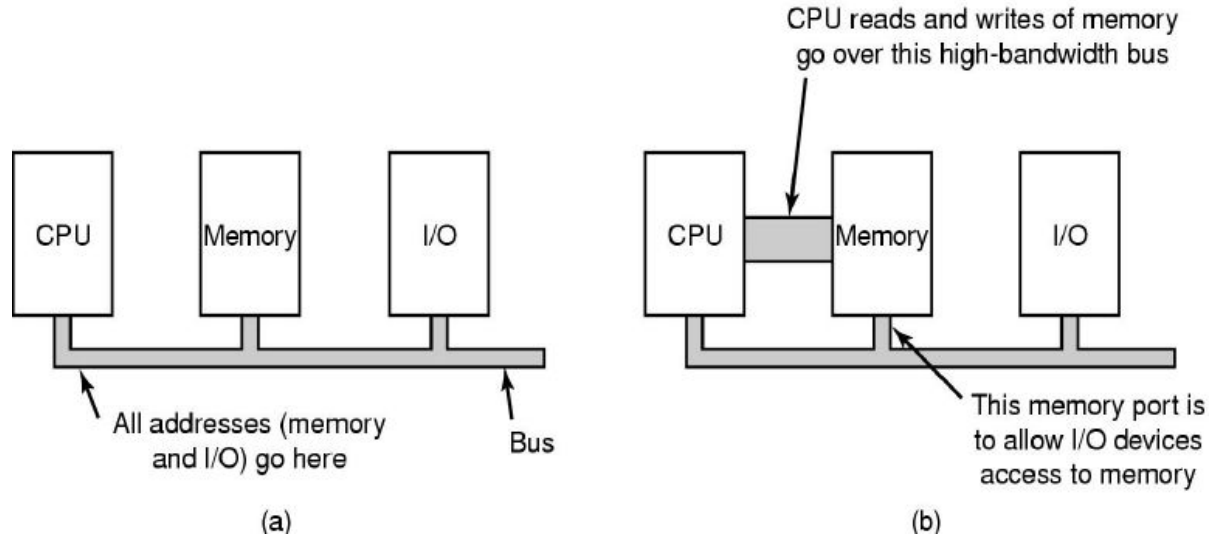


(a) Separate I/O and memory space

(b) Memory-mapped I/O

(c) Hybrid

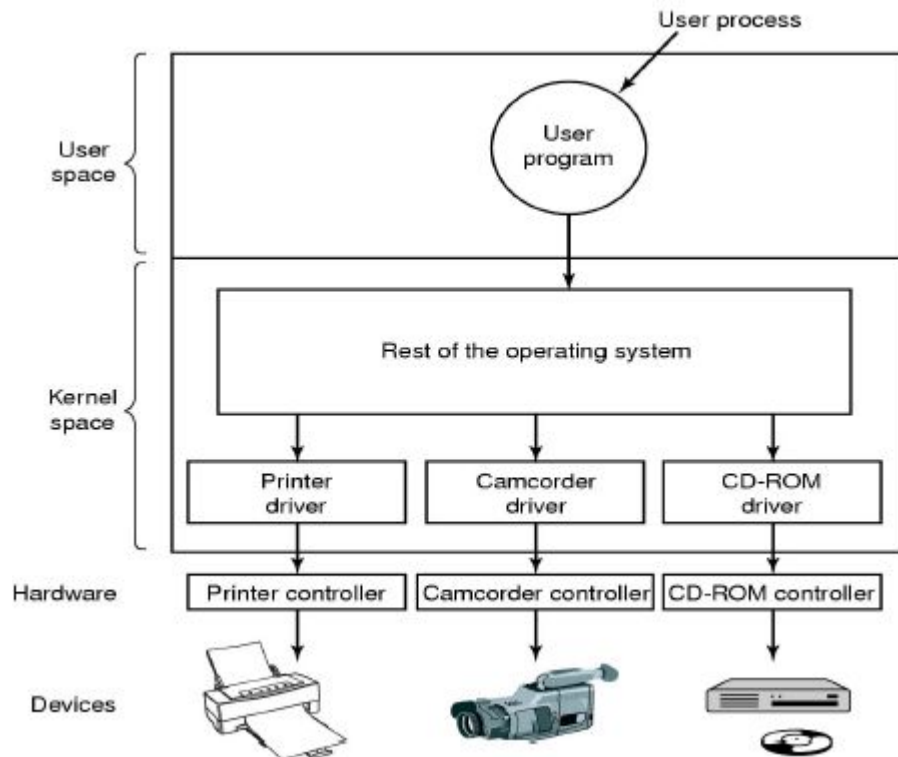
Memory-Mapped I/O (2)



(a) A single-bus architecture

(b) A dual-bus memory architecture

Device Drivers



- Logical position of device drivers is shown here
- Communications between drivers and device controllers goes over the bus
- Device-specific code to control an IO device, is usually written by device's manufacturer
 - Each controller has some device registers used to give it commands. The number of device registers and the nature of commands vary from device to device (e.g., mouse driver accepts information from the mouse how far it has moved, disk driver has to know about sectors, tracks, heads, etc).
- ☐ A device driver is usually part of the OS kernel
 - Compiled with the OS
 - Dynamically loaded into the OS during execution
- ☐ Each device driver handles
 - one device type (e.g., mouse)
 - one class of closely related devices (e.g., SCSI disk driver to handle multiple disks of different sizes and different speeds.).
- ☐ Categories:
 - ☐ Block devices
 - ☐ Character devices

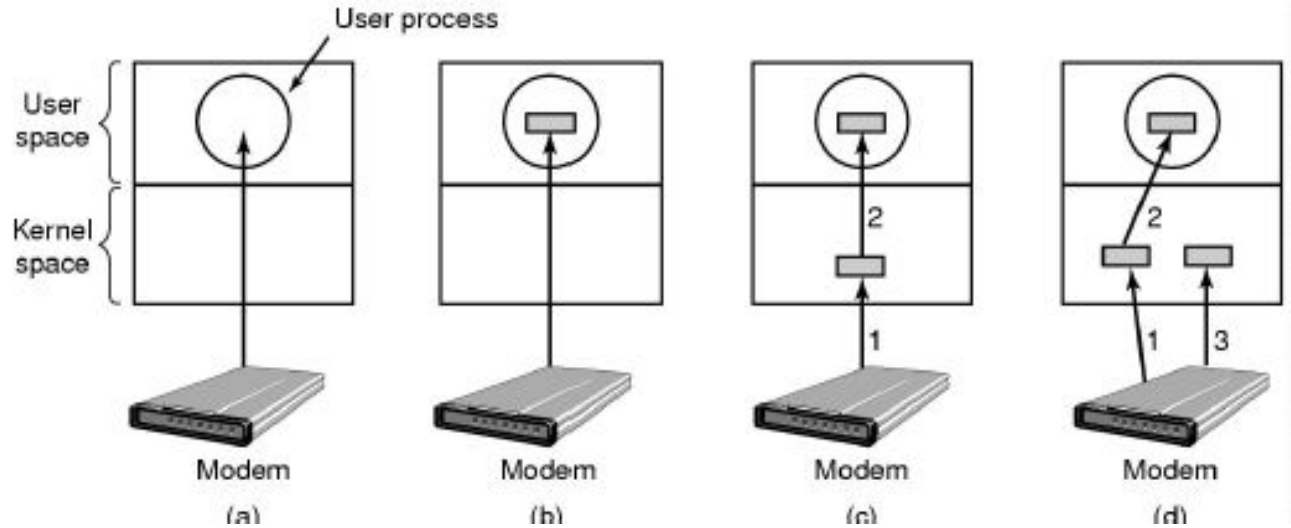
Functions in Device Drivers

- Accept abstract read and write requests from the device independent layer above;
- Initialize the device;
- Manage power requirements and log events
- Check input parameters if they are valid
- Translate valid input from abstract to concrete terms
 - e.g., convert linear block number into the head, track, sector and cylinder number for disk access
- Check the device if it is in use (i.e., check the status bit)
- Control the device by issuing a sequence of commands. The driver determines what commands will be issued.

Buffering

- Buffer is a memory area that stores data while they are transferred between two devices or between a device and an application.
- **Reasons of buffering:**
 - cope with speed mismatch between the producer and consumer of a data stream - use double buffering
 - adapt between devices that have different data-transfer sizes
 - support of copy semantics for application I/O - application writes to an application buffer and the OS copies it to the kernel buffer and then writes it to the disk.
 - Unbuffered input strategy is ineffective, as the user process must be started up with every incoming character.
 - Buffering is also important on output.
- **Caching**
 - cache is region of fast memory that holds copies of data and it allows for more efficient access.

Buffering strategies



(a) Unbuffered input

(b) Buffering in user space

(c) Buffering in the kernel followed by copying to user space

(d) Double buffering in the kernel

Error Reporting

- Programming IO Errors
 - these errors occur when a process asks for something impossible (e.g., write to an input device such as keyboard, or read from output device such as printer).
- Actual IO Errors
 - errors occur at the device level (e.g., read disk block that has been damaged, or try to read from video camera which is switched off)
- The device-independent IO software detects the errors and responds to them by reporting to the userspace IO software.